

PaperMentor 사용설명서

버전: 0.1.0

대상: Codex / Claude Code 사용자

문서 언어: 한국어

마지막 업데이트: 2026-06-25

1. PaperMentor가 하는 일

PaperMentor는 논문과 강의 슬라이드를 “요약”하는 도구가 아니라, 사용자가 실제로 내용을 따라갈 수 있도록 돕는 대화형 reading room 스킴이다.

핵심 목표는 다음과 같다.

- 논문이나 슬라이드의 현재 위치를 명확히 잡는다.
- 수식, 정의, 정리, 증명, 알고리즘, 그림을 원문 근거에 맞춰 설명한다.
- 사용자가 막힌 지점의 missing dependency를 찾아서 보수한다.
- 설명 결과를 터미널에 길게 뿌리지 않고, `index.html` reading room에 블록으로 축적한다.
- Codex와 Claude Code 양쪽에서 같은 방식으로 설치하고 사용할 수 있다.

PaperMentor가 특히 잘해야 하는 일은 다음과 같다.

- equation card: 수식 하나를 symbol by symbol로 해석한다.
- derivation trace: 수식에서 수식으로 넘어가는 연산을 한 단계씩 추적한다.
- proof walkthrough: 증명 줄 사이의 숨은 조작을 primitive operation level로 분해한다.
- dependency trace: 정의, 가정, 보조정리, 정리, 알고리즘 사이의 의존성을 연결한다.
- method dissection: method section의 실제 pipeline, objective, algorithm을 해부한다.
- slide mode: 강의 슬라이드를 timeline으로 읽고 빠진 강의 narration을 복원한다.
- representative figure reading: 논문의 대표 method/system/architecture figure를 실제 crop 이미지 기준으로 읽는다.

2. 설치 전 요구사항

필수 요구사항:

- Node.js 18 이상
- git
- macOS/Linux/Windows PowerShell 중 하나

PDF/PPT/image extraction을 제대로 쓰려면 아래 도구가 있으면 좋다.

- Poppler: `pdftotext`, `pdftoppm`
- LibreOffice: PPT/PPTX 변환용 `soffice`
- ImageMagick: crop/이미지 처리용 `magick` 또는 `convert`

설치 후 점검:

```
pm doctor
```

3. 설치 방법

3.1 Codex에 설치

```
curl -fsSL https://raw.githubusercontent.com/ShinyJay2/PaperMentor/main/install.sh | bash
```

설치 위치:

```
~/.codex/skills/papermentor
```

3.2 Claude Code에 설치

```
curl -fsSL https://raw.githubusercontent.com/ShinyJay2/PaperMentor/main/install.sh | bash -s -  
- claude
```

설치 위치:

```
~/.claude/skills/papermentor
```

3.3 Codex와 Claude Code에 동시에 설치

```
curl -fsSL https://raw.githubusercontent.com/ShinyJay2/PaperMentor/main/install.sh | bash -s -  
- all
```

이 경우 skill은 두 위치에 모두 설치되고, CLI wrapper는 한 번만 생성된다.

3.4 Windows PowerShell 설치

```
iwr -useb https://raw.githubusercontent.com/ShinyJay2/PaperMentor/main/install.ps1 -OutFile  
install.ps1  
.\install.ps1 codex  
.\install.ps1 claude  
.\install.ps1 all
```

3.5 로컬 clone 또는 pinned commit 설치

보안상 `main`에서 바로 pipe 실행하는 것이 부담스럽다면 다음 방식이 더 안전하다.

```
git clone https://github.com/ShinyJay2/PaperMentor.git  
cd PaperMentor  
./install.sh codex  
./install.sh claude  
./install.sh all
```

3.6 설치 옵션

CLI wrapper를 만들지 않기:

```
./install.sh all --no-cli
```

CLI wrapper 위치 지정:

```
./install.sh codex --bin-dir ~/.local/bin
```

Codex/Claude home 위치 지정:

```
CODEX_HOME=/custom/codex CLAUDE_HOME=/custom/claude ./install.sh all
```

4. 설치 과정 내부 구조

현재 설치 과정은 다음처럼 단순화되어 있다.

1. `install.sh` 또는 `install.ps1` 실행
2. 로컬 repo인지 확인
3. 원격 설치이면 `~/.papermentor/repo`에 shallow clone 또는 update
4. Node.js로 `scripts/install.mjs` 실행
5. `papermentor.manifest.json` 읽기
6. manifest에 적힌 runtime 파일만 Codex/Claude skill directory로 복사
7. `papermentor`와 `pm` CLI wrapper 생성

설치되는 runtime 파일 범위는 manifest가 관리한다.

```
skills/papermentor/**
prompts/**
templates/**
examples/**
tests/**
scripts/papermentor-session.mjs
assets/fonts/**
assets/mathjax/**
```

설치본에는 개발용 검증 스크립트, README 이미지, social preview, local session artifacts, PDF/PPT 소스 파일이 들어가지 않는다.

5. 가장 기본적인 사용법

논문 또는 슬라이드 파일로 reading room 시작:

```
pm paper.pdf
pm lecture.pdf
pm https://arxiv.org/pdf/2602.04770
```

현재/latest reading room HTML 열기:

```
pm open
```

화살표 기반 TUI로 계속 진행:

```
pm go
```

질문하기:

```
pm ask "Eq. (3)에서 왜 expectation으로 바뀌어?"
pm ask "이 proof line 사이에 어떤 조작이 생략된 거야?"
```

품질 검사:

```
pm qa
```

PDF export:

pm export

최근 reading room 목록:

pm recent

6. Reading room의 파일 구조

PaperMentor는 소스마다 하나의 session folder를 만든다.

```
.papermentor/sessions/<slug>/  
index.html  
cards.json  
state.json  
turns.jsonl  
notes.md  
assets/
```

중요한 파일:

- `index.html`: 사용자가 보는 최종 reading room
- `cards.json`: HTML 블록 데이터
- `state.json`: 현재 위치, 선택지, 자동 생성/agent handoff 상태
- `turns.jsonl`: 대화 로그
- `notes.md`: 사람이 읽기 쉬운 note snapshot
- `assets/`: figure crop, fonts, MathJax, generated diagram assets

HTML은 local fonts와 local MathJax를 사용한다. 세션 asset은 hardlink-first로 연결하고, 파일시스템이 지원하지 않으면 copy로 fallback한다.

7. Paper mode 사용 흐름

Paper mode는 연구 논문을 읽기 위한 기본 모드다.

시작하면 PaperMentor는 다음을 만든다.

1. `How to use this reading room` 블록
2. `Start Here` 블록. Codex/Claude 자동화가 가능하면 바로 채워지고, 아니면 agent가 이어서 완성할 내부 handoff 상태
3. source section navigator
4. representative figure 후보 선택 상태, 필요한 경우

Paper mode의 주요 action:

- Map the paper
- Decode key equations
- Trace derivations

- Connect dependencies
- Resolve confusion
- Extract final insight

수식이나 증명을 볼 때 중요한 원칙:

- LaTeX notation을 유지한다.
- symbol table을 만든다.
- 수식 전환을 한 줄씩 추적한다.
- 생략된 가정과 정의를 드러낸다.
- proof에서는 line transition microscope처럼 줄 사이 연산을 primitive operation level로 분해한다.

8. Slide mode 사용 흐름

Slide mode는 강의 슬라이드 PDF/PPT를 위한 모드다.

명시적으로 slide mode로 열기:

```
pm lecture.pdf --mode slide
```

Slide mode는 논문처럼 section 중심으로 읽지 않는다. 슬라이드는 temporal timeline이므로 다음을 중요하게 본다.

- slide title과 slide image
- 같은 제목이 반복되는 build slide 묶기
- topic timeline map
- 이전 slide에서 다음 slide로 넘어가는 논리
- 강사가 말했을 likely missing narration
- on-slide equation과 visual 강조

Slide mode의 Start Here는 paper 대표그림 레이아웃이 아니라 topic timeline map을 중심으로 만들어진다.

9. 대표 그림 선택 방식

PaperMentor는 대표 그림을 script score로 고르지 않는다.

현재 방식:

1. PDF text에서 Figure 1:, Fig. 2. 같은 caption-looking 후보만 구조적으로 수집한다.
2. caption text, page, section context, nearby text를 모은다.
3. representative-figure-prompt.md를 만든다.
4. LLM이 semantic하게 대표 method/system/architecture figure인지 판단한다.

5. result plot, benchmark, ablation이면 selected: none을 선택할 수 있다.

6. 선택된 crop은 pixel 기준으로 다시 읽는다.

즉 hard-coded keyword score가 아니라, prompt가 후보를 보고 판단한다.

대표 그림 설명은 caption만 읽고 쓰면 안 된다. crop 이미지를 열어 실제로 보이는 box, arrow, line, color, axis, label, in-figure math를 하나씩 읽어야 한다.

10. Proof / derivation 설명 원칙

증명과 유도에서 PaperMentor가 해야 하는 핵심은 “줄 사이에 무슨 일이 일어났는지”를 설명하는 것이다.

좋은 proof explanation은 다음을 포함한다.

- 현재 claim이 무엇인지
- 각 symbol이 무엇을 뜻하는지
- proof line 1에서 proof line 2로 갈 때 어떤 operation이 있었는지
- 여러 operation이 한 번에 일어났다면 모두 primitive step으로 쪼개기
- 어떤 정의, lemma, assumption, algebraic identity, probability rule이 쓰였는지
- 왜 그 조작이 valid한지
- 그 line transition이 최종 claim에 어떻게 가까워지는지

중요한 점:

- 특정 operation list에 갇히면 안 된다.
- 실제 paper에서 사용된 operation을 source에서 읽어야 한다.
- matrix multiplication, conditional expectation, indicator restriction, inequality, limit exchange 등 무엇이든 실제 줄 사이에 있으면 설명해야 한다.
- 설명은 표 형식에 고정되지 않아도 된다. 핵심은 line transition microscope다.

11. CLI command reference

사용자용 간단 명령:

pm	# 메인 메뉴 열기
pm <file-or-url>	# reading room 시작
pm open	# HTML 열기
pm go	# TUI 계속
pm ask "question"	# 질문하기
pm qa	# teaching quality 검사
pm export	# PDF export
pm recent	# 최근 session 목록
pm doctor	# 로컬 도구 확인

고급/내부 명령:

```
papermentor launch <source> [--mode paper|slide] [--slug <slug>]
papermentor tui --session <slug>
papermentor run --session <slug> --index <n>
```

```
papermentor card --session <slug> --type equation --title <title> --body-file answer.md
papermentor preview-crops --session <slug> --source paper.pdf --page 1
papermentor extract-figure --session <slug> --source paper.pdf --page 1 --auto figure1
papermentor qa --session <slug>
papermentor export --session <slug> --format pdf
papermentor doctor --json
```

일반 사용자는 대부분 `pm` 명령만 쓰면 된다.

12. 품질 검사와 배포 검증

개발자용 검증:

```
npm test
npm run manifest:check
npm run pack:check
```

각 명령의 의미:

- `npm test`: runtime behavior, rendering, source mode, QA, install smoke 검사
- `npm run manifest:check`: `package.json` files와 `papermentor.manifest.json` 동기화 검사
- `npm run pack:check`: npm package 내용, forbidden files, 용량, 파일 수 검사

패키지에 들어가면 안 되는 것:

- `.papermentor/`
- `.omx/`, `.omc/`
- `.claude/`
- local screenshot
- root PDF/PPT/PPTX
- `image.png`
- social preview PNG

13. Troubleshooting

`pm` 명령을 찾을 수 없음

설치 메시지에 나온 bin directory가 PATH에 없을 수 있다.

macOS/Linux 기본 위치:

```
~/local/bin
```

해결:

```
export PATH="$HOME/local/bin:$PATH"
```

PDF text extraction이 안 됨

Poppler가 없을 수 있다.

macOS:

```
brew install poppler
```

PPT/PPTX가 안 열림

LibreOffice가 필요할 수 있다.

```
brew install --cask libreoffice
```

HTTP URL이 막힘

PaperMentor는 기본적으로 HTTPS source만 다운로드한다. 테스트용 local HTTP server는 명시적으로 허용해야 한다.

```
pm http://127.0.0.1:8000/paper.pdf --allow-insecure-http
```

HTML은 보이는데 수식이 안 렌더링됨

세션 assets에 MathJax가 있어야 한다.

```
ls .papermentor/sessions/<slug>/assets/mathjax/tex-svg.js
```

없으면 session을 다시 render하거나 재설치한다.

```
papermentor render --session <slug>  
./install.sh codex
```

14. 좋은 사용 패턴

좋은 질문:

Eq. (6)에서 stopgrad가 왜 필요한지 설명해줘.
이 proof에서 두 번째 줄에서 세 번째 줄로 갈 때 어떤 항이 expectation 안으로 들어간 거야?
이 lemma가 theorem에 어떻게 쓰이는지 dependency trace 해줘.
이 slide에서 강사가 말했을 narration을 복원해줘.

덜 좋은 질문:

이 논문 요약해줘.
대충 핵심만 알려줘.
그림을 새로 예쁘게 그려줘.

PaperMentor의 강점은 “대충 요약”이 아니라 “내가 막힌 지점을 고쳐서 다시 읽을 수 있게 만드는 것”이다.

15. 운영 원칙

PaperMentor를 수정할 때 지켜야 할 원칙:

1. semantic 판단은 script가 아니라 prompt가 하게 한다.
2. script는 구조 수집, 파일 처리, rendering, session state에 집중한다.
3. hard-coded keyword matching으로 topic/figure/proof 의미를 결정하지 않는다.
4. HTML block은 사용자가 바로 읽을 수 있는 teaching output이어야 한다.

5. CLI는 긴 설명을 뿌리는 곳이 아니라 navigation, 자동 생성 진행 상태, HTML 링크를 보여주는 곳이다. 내부 prompt 파일은 일반 사용자 단계로 노출하지 않는다.
6. 설치/패키징 파일 목록은 `papermentor.manifest.json`에서 먼저 수정한다.
7. 수정 후 반드시 `npm test`, `npm run manifest:check`, `npm run pack:check`를 실행한다.

16. 빠른 시작 요약

설치:

```
curl -fsSL https://raw.githubusercontent.com/ShinyJay2/PaperMentor/main/install.sh | bash -s -  
- all
```

논문 열기:

```
pm paper.pdf
```

HTML 열기:

```
pm open
```

질문하기:

```
pm ask "이 수식에서 왜 이 항이 사라져?"
```

계속 진행:

```
pm go
```

품질 검사:

```
pm qa
```

PDF export:

```
pm export
```